QUIZ Pertemuan 6 — Abstraction dan Interface pada Python

Nama	**:**	**Jihan Maharani**
NIM	**:**	**250660221031**
Kelas	**:**	**Sistem Informasi 3-A**
Mata Kuliah	**:**	**Pemrograman Berorientasi Objek**

SOAL

1. **Soal 1 — Konsep Abstraction**
   Jelaskan pengertian abstraction dalam OOP dan berikan satu contoh antarmuka yang menyembunyikan detail implementasi.

   Jawaban:
   Abstraction adalah konsep OOP yang menyembunyikan detail proses atau implementasi di dalam program dan hanya menampilkan fungsi penting yang perlu diketahui oleh pengguna.
   Contohnya adalah mesin ATM. Pengguna cukup memilih menu tarik tunai dan memasukkan jumlah uang tanpa perlu mengetahui proses teknis di dalam sistem ATM.

2. **Soal 2 — Abstract Class dan Abstract Method**
   Jelaskan perbedaan abstract class dan abstract method. Mengapa abstract class tidak dapat diinstansiasi langsung?

   Jawaban:
   Abstract class adalah class dasar yang digunakan sebagai kerangka bagi class turunannya. Class ini tidak dapat dibuat menjadi object secara langsung jika masih memiliki abstract method yang belum diimplementasikan.
   Abstract method adalah method yang hanya dideklarasikan pada abstract class dan harus diimplementasikan oleh subclass.
   Abstract class tidak dapat diinstansiasi secara langsung karena Python mengharuskan semua abstract method diimplementasikan terlebih dahulu oleh subclass.

3. **Soal 3 — Prediksi Output**

```
from abc import ABC, abstractmethod

class Pengguna(ABC):
    def __init__(self, nama):
        self.nama = nama

    @abstractmethod
    def tampilkan_peran(self):
        pass
```

```
class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"

print(Mahasiswa("Budi").tampilkan_peran())
```

Tuliskan output dari program tersebut dan jelaskan peran @abstractmethod.

Jawaban:
Output:
Mahasiswa

Penjelasan:
Class Mahasiswa berhasil dibuat karena sudah mengimplementasikan abstract method tampilkan_peran() dari class Pengguna.
@abstractmethod berfungsi untuk menandai method sebagai method abstrak yang wajib diimplementasikan oleh subclass sebelum object dapat dibuat.

4. Soal 4 — Error pada Abstract Class

```
from abc import ABC, abstractmethod

class Pengguna(ABC):
    @abstractmethod
    def tampilkan_peran(self):
        pass

class Mahasiswa(Pengguna):
    pass

mahasiswa = Mahasiswa()
```

Apa yang terjadi ketika Mahasiswa() dijalankan? Jelaskan alasan teknisnya.

Jawaban:
Kode tersebut akan menghasilkan TypeError ketika Mahasiswa() dijalankan.
Alasan teknis:
Class Mahasiswa merupakan subclass dari Pengguna, tetapi belum mengimplementasikan abstract method tampilkan_peran().
Karena method abstrak tersebut belum diimplementasikan, Python tidak mengizinkan pembuatan object dari class Mahasiswa.

5. Soal 5 — Merancang Abstract Class

Tuliskan abstract class Transaksi dengan abstract method proses(), lalu buat subclass TransaksiTunai yang mengimplementasikan method tersebut. Jelaskan manfaat kontrak perilaku pada rancangan tersebut.

Jawaban:
Berikut contoh abstract class Transaksi dan subclass TransaksiTunai:

from abc import ABC, abstractmethod

class Transaksi(ABC):

```
    @abstractmethod
    def proses(self):
        pass

class TransaksiTunai(Transaksi):
    def proses(self):
        return "Transaksi tunai berhasil diproses"

transaksi = TransaksiTunai()
print(transaksi.proses())
```

Output:
Transaksi tunai berhasil diproses

Penjelasan:
Class Transaksi menjadi abstract class yang memiliki abstract method proses(). Class TransaksiTunai bertugas mengimplementasikan method tersebut sesuai dengan kebutuhan transaksi tunai.
Manfaat kontrak perilaku adalah memastikan setiap subclass memiliki method proses() sehingga struktur program lebih konsisten, terorganisasi, dan mudah dikembangkan.